DOSSIER PROJET

Titre professionnel : **Développeur Web et Web Mobile**

Projet : **Knowledge Learning - Plateforme e-learning/e-commerce**

Nom : **ESSOUFI**

Prénom : **RIM**

Adresse : **7 rue Marcel Pagnol, 31100 Toulouse**

Centre Européen de Formation

Année : 2026

REMERCIEMENTS

Je tiens à remercier le Centre Européen de Formation pour la qualité de l'enseignement dispensé tout au long de cette formation.

Je remercie également mes formateurs et mon mentor pour leur accompagnement
et leurs conseils précieux.

Enfin, je remercie ma famille et mes proches pour leur soutien constant durant cette année de formation.

SOMMAIRE

4. Liste des compétences du référentiel couvertes par le projet

5. Résumé du projet en français

6. Cahier des charges / Expression des besoins

7. Spécifications techniques du projet

9. Réalisations du candidat (extraits de code)

16. Présentation du jeu d'essai

17. Description de la veille sur les vulnérabilités de sécurité

18. Description d'une situation de travail ayant nécessité une recherche à partir de sites anglophones

19. Extrait du site anglophone + traduction en français

1. LISTE DES COMPÉTENCES DU RÉFÉRENTIEL COUVERTES PAR LE PROJET

Activité-type 1 : Développer la partie front-end d'une application web ou web mobile sécurisée

- Maquetter des interfaces utilisateur web ou web mobile
- Réaliser des interfaces utilisateur statiques web ou web mobile
- Développer la partie dynamique des interfaces utilisateur web ou web mobile

Activité-type 2 : Développer la partie back-end d'une application web ou web mobile sécurisée

- Mettre en place une base de données relationnelle
- Développer des composants d'accès aux données SQL et NoSQL
- Développer des composants métier côté serveur

Compétence transversale complémentaire :

- Documenter le déploiement d'une application dynamique web ou web mobile

2. RÉSUMÉ DU PROJET

Knowledge Learning est une plateforme e-learning/e-commerce développée avec Symfony 8.1.

L'entreprise Knowledge, spécialisée dans l'édition de livres de formation, souhaitait développer son offre en proposant des formations en ligne.

Le projet a été réalisé en suivant une architecture MVC (Modèle-Vue-Contrôleur). La base de données a été conçue avec la méthode Merise et comprend 8 tables principales : utilisateurs, rôles, thèmes, cursus, leçons, achats, validations et certifications.

Les fonctionnalités principales incluent : inscription avec activation par email (via Mailtrap), authentification sécurisée avec rôles (admin/client), achat de cursus et de leçons via Stripe (sandbox), validation des leçons avec obtention automatique de certifications.

La sécurité a été renforcée avec la protection CSRF, le hachage des mots de passe (bcrypt), et la validation des entrées utilisateur.

Des tests unitaires (PHPUnit) ont été réalisés pour valider les fonctionnalités essentielles : création de compte, connexion, achat et accès aux données.

Le projet est disponible sur GitHub et fonctionne en local.

Un back-office admin sur le site permet de gérer les utilisateurs et de visualiser les statistiques.

3. CAHIER DES CHARGES / EXPRESSION DES BESOINS

3.1. Contexte du projet

L'entreprise Knowledge souhaite créer une plateforme e-learning permettant à ses clients d'acheter et de suivre des formations en ligne.

3.2. Objectifs

- Permettre aux utilisateurs de créer un compte et de l'activer par email
- Permettre aux utilisateurs d'acheter des cursus et des leçons
- Permettre aux utilisateurs de valider des leçons
- Délivrer automatiquement des certifications après validation
- Offrir un back-office administrateur pour gérer les utilisateurs et les contenus

3.3. Contraintes techniques

- Langage : PHP 8.4
- Framework : Symfony 8.1
- Base de données : MySQL
- Paiement : Stripe (mode test)
- Email : Mailtrap (sandbox)
- Versionnage : Git/GitHub

3.4. Livrables attendus

- Application fonctionnelle accessible localement
- Code source sur GitHub (public)
- Documentation technique
- Tests unitaires

4. SPÉCIFICATIONS TECHNIQUES DU PROJET

4.1. Architecture MVC

- Modèle : Doctrine ORM (entités et repositories)
- Vue : Twig (templates)
- Contrôleur : PHP (Symfony)

4.2. Base de données (8 tables)

- roles : rôles des utilisateurs (admin, client)
- users : utilisateurs (email, password, is_active, is_verified, activation_token, role_id)
- themes : thèmes de formation (Musique, Informatique, Jardinage, Cuisine)
- courses : cursus (title, price, description, theme_id)
- lessons : leçons (title, price, content, video_url, order_number, course_id)
- purchases : achats (user_id, purchasable_type, purchasable_id, amount, payment_status, stripe_payment_id)
- user_lessons : table de jointure N-N (user_id, lesson_id, is_validated, validated_at)
- certifications : certifications obtenues (user_id, course_id, obtained_at)

4.3. Relations principales

- Role (1) ↔ User (N)
- Theme (1) ↔ Course (N)
- Course (1) ↔ Lesson (N)
- User (N) ↔ Lesson (N) via user_lessons
- User (1) ↔ Purchase (N)
- User (1) ↔ Certification (N)
- Course (1) ↔ Certification (N)

4.4. Sécurité

- Authentification avec Symfony Security Bundle
- Hachage des mots de passe avec bcrypt
- Protection CSRF sur tous les formulaires
- Protection XSS avec Twig {{ }}
- Protection SQL Injection avec requêtes préparées Doctrine
- Validation des entrées utilisateur
- Activation par email avec token unique

4.5. Front-end

- Framework CSS : Bootstrap 5.3
- Responsive design avec media queries
- Menu hamburger sur mobile
- Twig pour le dynamisme

4.6. Accessibilité

Pour garantir l'accessibilité de l'application, j'ai mis en place :

- Contrastes suffisants entre textes et fonds
- Navigation au clavier
- Attributs alt sur les images
- Balises HTML sémantiques (header, nav, main, footer)
- Respect des normes WCAG

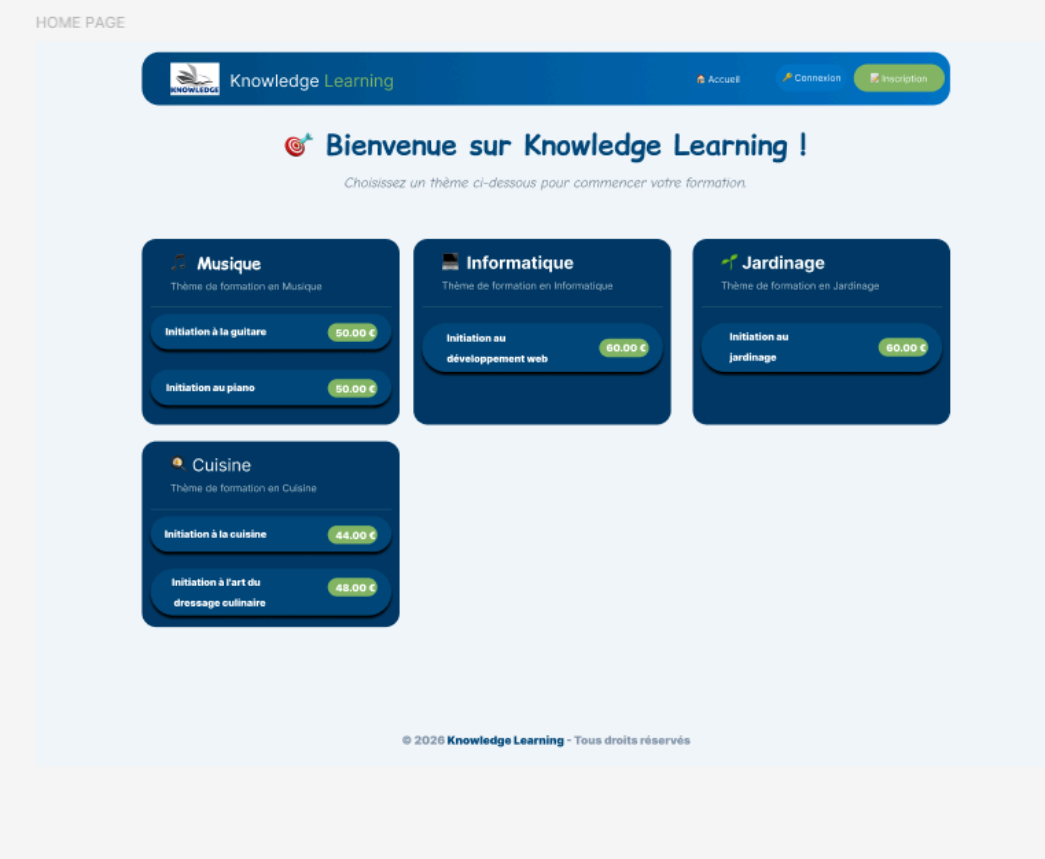
4.7. Optimisation SEO

J'ai optimisé le référencement de l'application avec :

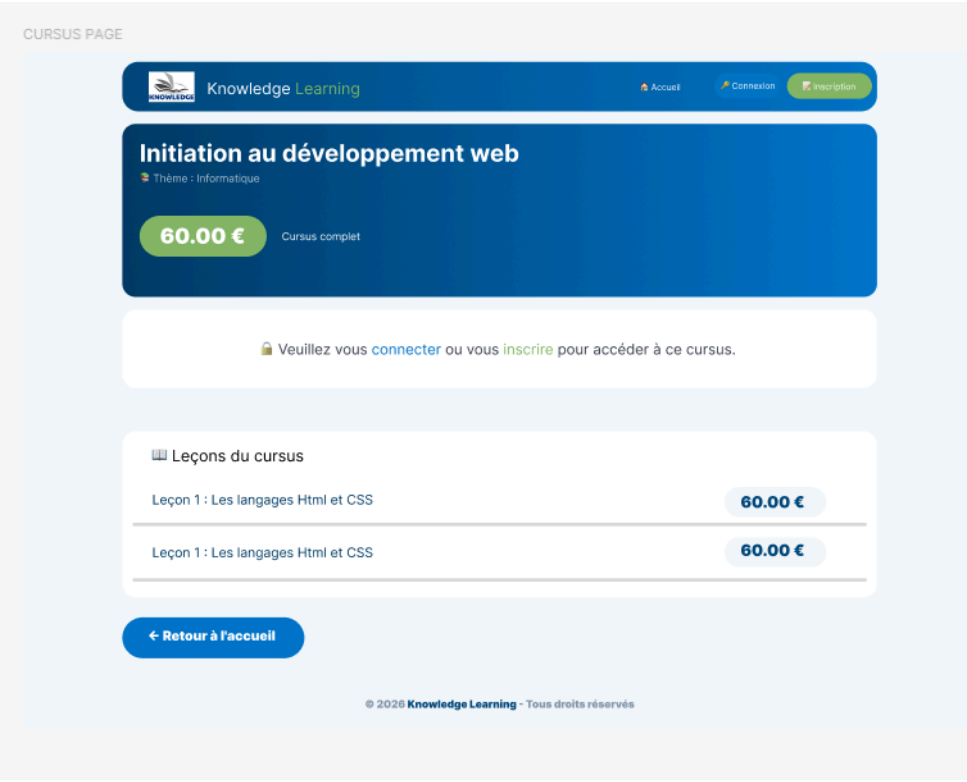
- Balises meta (title, description) uniques par page
- URLs lisibles (ex: /cursus/initiation-guitare)
- Structure des titres hiérarchisée (h1, h2, h3)
- Attributs alt sur les images
- Site responsive (mobile-friendly)
- Temps de chargement optimisé

4.8. Maquettage

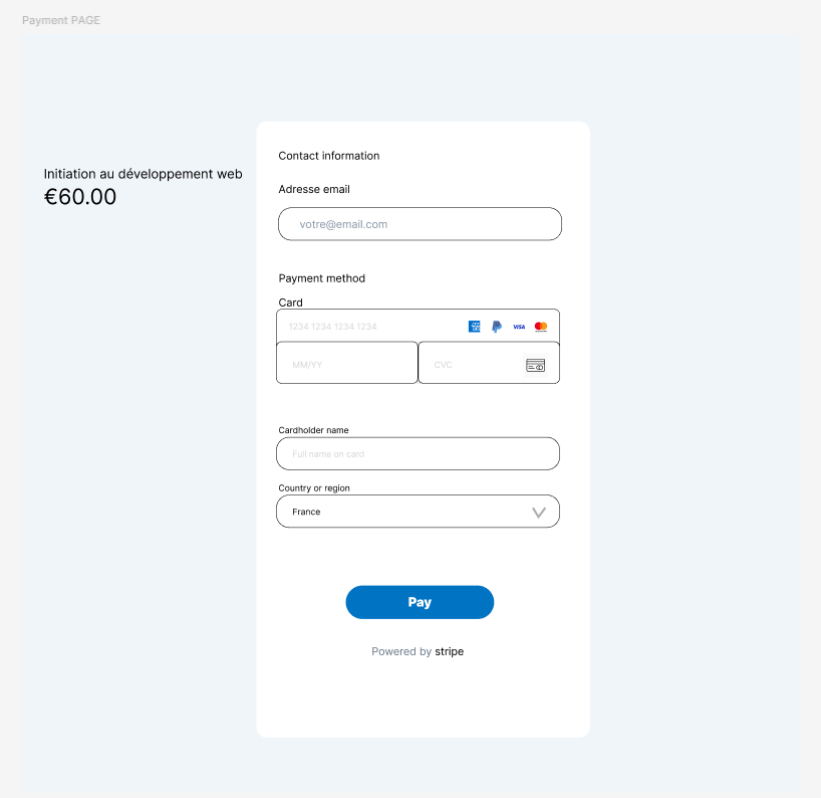
Maquette - Page d'accueil



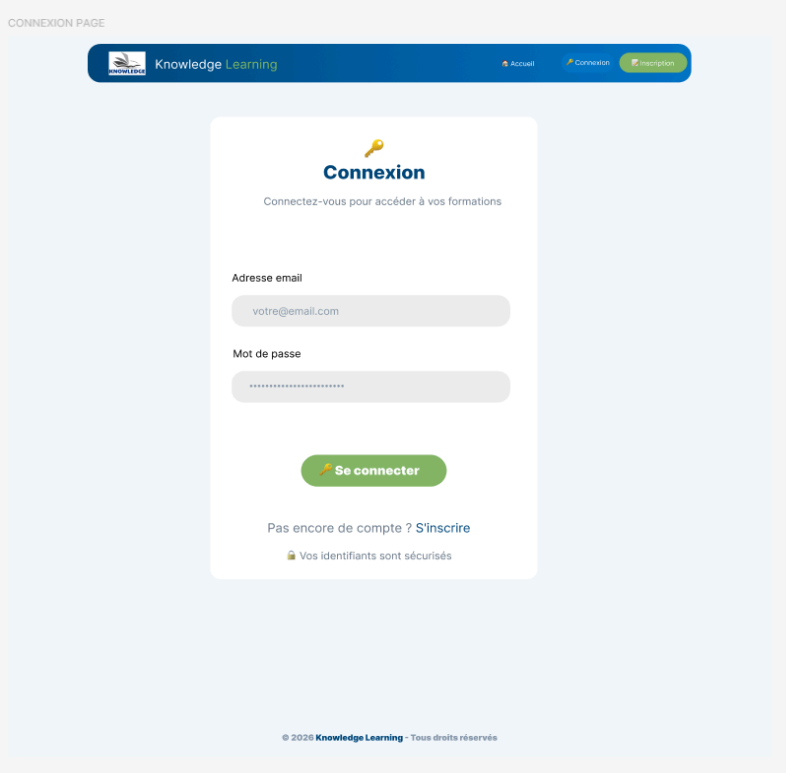
Maquette - Page Cursus



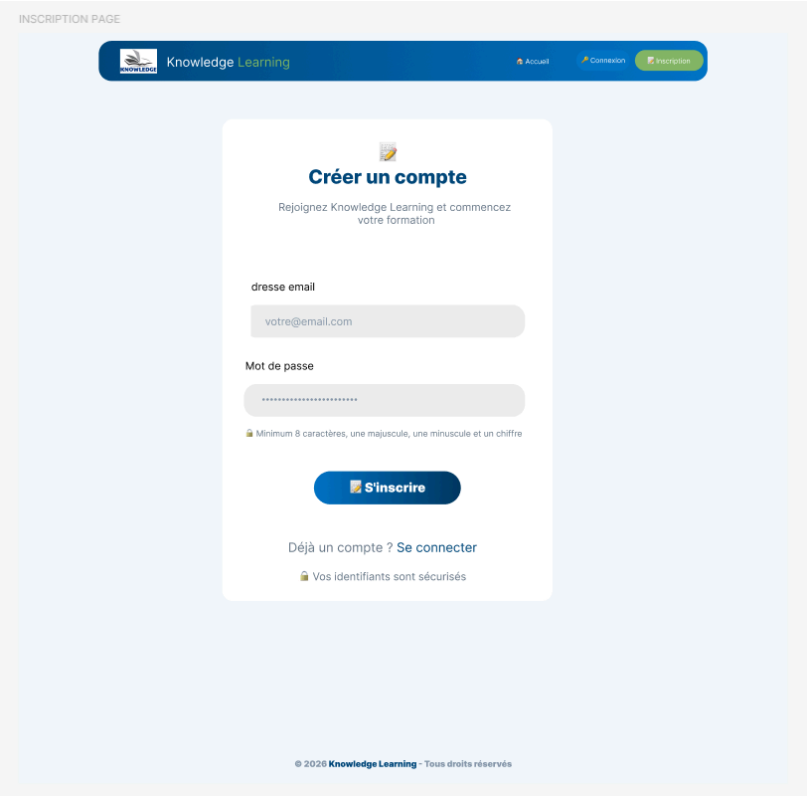
Maquette - Page Panier



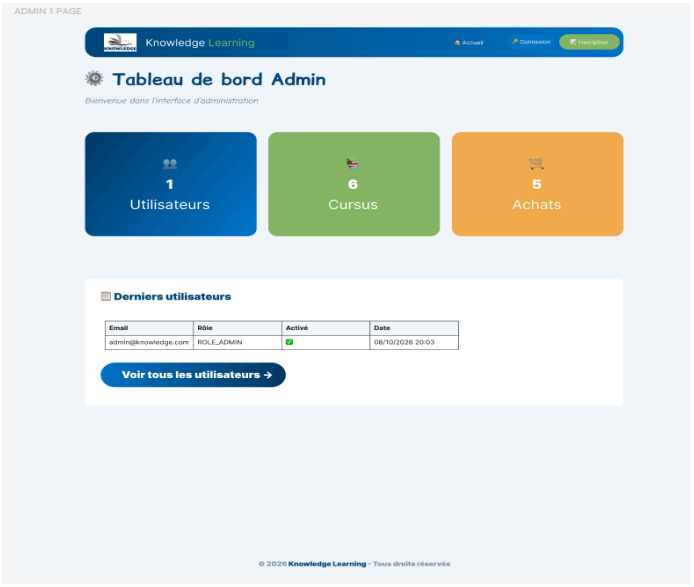
Maquette - Page Connexion

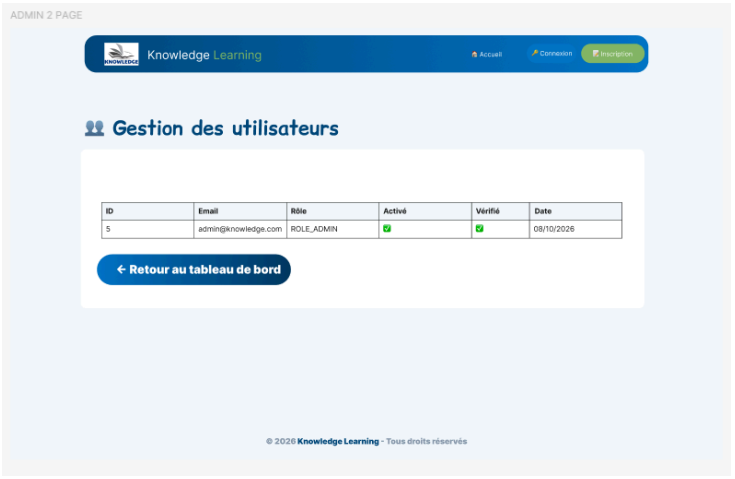


Maquette - Page Inscription



Maquette - Admin Dashboard





- Le prototype est disponible sur Figma :

https://www.figma.com/proto/3HNXCUPbOPIYQrk8NhDRf9/Knowledge-Learning-WIREFRAME?node-id=0-1&t=wk0HNewFVP2m9Fds-1

5. RÉALISATIONS DU CANDIDAT (EXTRAITS DE CODE)

5.1. Entité User (Doctrine)

src/Entity/User.php

```php
#[ORM\Entity(repositoryClass: UserRepository::class)]
#[UniqueEntity(fields: ['email'], message: 'There is already an account with this email')]
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 180, unique: true)]
    private ?string $email = null;

    #[ORM\Column]
    private ?string $password = null;

    #[ORM\Column]
    private bool $isActive = false;

    #[ORM\Column]
    private bool $isVerified = false;

    #[ORM\Column(length: 100, nullable: true)]
    private ?string $activationToken = null;

    #[ORM\ManyToOne]
    #[ORM\JoinColumn(nullable: false)]
    private ?Role $role = null;
```

5.2. Contrôleur d'inscription (RegistrationController)

src/Controller/RegistrationController.php

```
#[Route('/register', name: 'app_register')]

public function register(Request $request, UserPasswordHasherInterface
$userPasswordHasher, EntityManagerInterface $entityManager): Response

{

    $user = new User();

    $form = $this->createForm(RegistrationFormType::class, $user);

    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {

        $user->setPassword(

            $userPasswordHasher->hashPassword(

                $user,

                $form->get('plainPassword')->getData()

            )

        );

        $entityManager->persist($user);

        $entityManager->flush();

              $this->emailVerifier->sendEmailConfirmation('app_verify_email',
$user,

            (new TemplatedEmail())

                    ->from(new Address('knowledge@learning.com', 'Knowledge
Learning'))

                ->to($user->getEmail())

                ->subject('Confirmez votre email')

                ->htmlTemplate('registration/confirmation_email.html.twig')

        );

        return $this->redirectToRoute('app_home');

    }

    return $this->render('registration/register.html.twig', [

        'registrationForm' => $form->createView(),

    ]); }
```

5.3. Template Twig (page d'accueil)

templates/home/index.html.twig

```
{% extends 'base.html.twig' %}


{% block body %}
    <h1>Bienvenue sur Knowledge Learning !</h1>
    <p>Choisissez un thème ci-dessous pour commencer votre formation.</p>


    <div class="row">
        {% for theme in themes %}
            <div class="col-md-3 mb-4">
                <div class="card">
                    <div class="card-body">
                        <h5 class="card-title">{{ theme.name }}</h5>
                        <p class="card-text">{{ theme.description }}</p>
                           <a href="{{ path('app_theme_show', {id: theme.id})
}}" class="btn btn-primary">Voir le thème</a>
                    </div>
                </div>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

5.4. Repository personnalisé (ProductRepository)

src/Repository/UserRepository.php

```php
class UserRepository extends ServiceEntityRepository
{
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, User::class);
    }

    public function findActiveUsers(): array
    {
        return $this->createQueryBuilder('u')
            ->andWhere('u.isActive = :active')
            ->setParameter('active', true)
            ->orderBy('u.createdAt', 'DESC')
            ->getQuery()
            ->getResult();
    }
}
```

5.5. Configuration de sécurité (security.yaml)

config/packages/security.yaml

```yaml
security:
    password_hashers:

Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
'auto'
    providers:
        app_user_provider:
            entity:
                class: App\Entity\User
                property: email
    firewalls:
        main:
            lazy: true
            provider: app_user_provider
            form_login:
                login_path: app_login
                check_path: app_login
                enable_csrf: true
            logout:
                path: app_logout
    access_control:
        - { path: ^/admin, roles: ROLE_ADMIN }
        - { path: ^/profile, roles: ROLE_USER }
```

5.6. Service Stripe (StripeService.php)

src/Service/StripeService.php

```
class StripeService
{
    public function __construct(string $secretKey, string $publishableKey)
    {
        Stripe::setApiKey($secretKey);
    }

      public function createCheckoutSession(array $cart, string $successUrl,
string $cancelUrl): Session
    {
        $lineItems = [];
        foreach ($cart as $item) {
            $lineItems[] = [
                'price_data' => [
                    'currency' => 'eur',
                    'product_data' => ['name' => $item['name']],
                    'unit_amount' => (int)($item['price'] * 100),
                ],
                'quantity' => 1,
            ];
        }

        return Session::create([
            'payment_method_types' => ['card'],
            'line_items' => $lineItems,
            'mode' => 'payment',
            'success_url' => $successUrl,
            'cancel_url' => $cancelUrl,
        ]);
    }
}
```

5.7. Tests unitaires (PHPUnit)

tests/Repository/UserRepositoryTest.php

```php
class UserRepositoryTest extends TestCase
{
    public function testFindActiveUsers(): void
    {
        $user = new User();
        $user->setEmail('test@example.com');
        $user->setIsActive(true);

        $this->assertTrue($user->isActive());
        $this->assertEquals('test@example.com', $user->getEmail());
    }

    public function testGetRoles(): void
    {
        $user = new User();
        $this->assertContains('ROLE_CLIENT', $user->getRoles());
    }
}
```

6. PRÉSENTATION DU JEU D'ESSAI

6.1. Fonctionnalité testée : Inscription d'un utilisateur

Données en entrée :

- Email : test@example.com
- Mot de passe : Password123!
- Confirmation : Password123!

Données attendues :

- L'utilisateur est créé en base de données
- Le mot de passe est haché (bcrypt)
- Un email de confirmation est envoyé à test@example.com
- L'utilisateur est redirigé vers la page d'accueil

Données obtenues :

- L'utilisateur est bien créé en base
- Le mot de passe est bien haché
- L'email est bien envoyé (visible dans Mailtrap)
- La redirection fonctionne

Analyse des écarts : Aucun écart constaté.

6.2. Fonctionnalité testée : Achat d'un cursus

Données en entrée :

- Utilisateur connecté
- Clic sur "Acheter ce cursus"
- Redirection Stripe

Données attendues :

- Session de paiement Stripe créée
- Redirection vers Stripe
- Paiement simulé (carte test 4242 4242 4242 4242)
- Retour sur le site avec confirmation

Données obtenues :

- Session Stripe créée avec succès
- Redirection vers Stripe fonctionnelle
- Paiement test validé
- Statut de la commande mis à jour en BDD

Analyse des écarts : Aucun écart constaté.

7. DESCRIPTION DE LA VEILLE SUR LES VULNÉRABILITÉS DE SÉCURITÉ

7.1. Sites consultés

- OWASP (Open Web Application Security Project) - owasp.org
- ANSSI (Agence Nationale de la Sécurité des Systèmes d'Information) - ssi.gouv.fr
- CERT-FR (Centre gouvernemental de veille, d'alerte et de réponse aux attaques informatiques) - cert.ssi.gouv.fr

7.2. Vulnérabilités identifiées

- XSS (Cross-Site Scripting) : injection de scripts malveillants dans les pages web
- CSRF (Cross-Site Request Forgery) : falsification de requête pour exécuter des actions non désirées
- SQL Injection : injection de code SQL dans les requêtes
- Mots de passe faibles : stockage en clair ou hachage insuffisant

7.3. Failles corrigées dans mon projet

- XSS → Échappement automatique avec Twig {{ }}
- CSRF → Token unique généré pour chaque formulaire
- SQL Injection → Requêtes préparées avec Doctrine
- Mots de passe → Hachage bcrypt via UserPasswordHasherInterface
- Validation → Contraintes sur les entités (email unique, mot de passe fort)
- Session → Configuration sécurisée de Symfony

8. DESCRIPTION D'UNE SITUATION DE TRAVAIL AYANT NÉCESSITÉ UNE RECHERCHE À PARTIR DE SITES ANGLOPHONES

Besoin d'information :

J'avais besoin de configurer l'envoi d'emails avec Mailtrap pour l'activation des comptes utilisateurs. Je ne savais pas comment configurer le DSN dans le fichier .env de Symfony.

Mots-clés utilisés :

- "symfony mailer mailtrap configuration"
- "symfony mailer DSN example"
- "how to configure mailtrap with symfony"

Sites consultés :

- Stack Overflow (stackoverflow.com)
- Symfony Documentation (symfony.com/doc)
- Mailtrap Documentation (mailtrap.io)

Critères de sélection :

J'ai choisi les sites officiels (Symfony, Mailtrap) et Stack Overflow pour la qualité des réponses et la communauté active.

Solution trouvée :

La configuration correcte dans le fichier .env est :

MAILER_DSN=smtp://username:password@sandbox.smtp.mailtrap.io:2525

Mise en œuvre :

J'ai appliqué cette configuration et l'envoi d'emails a fonctionné immédiatement. J'ai pu tester l'activation de compte avec succès.

9. EXTRAIT DU SITE ANGLOPHONE + TRADUCTION

Extrait original (anglais) :

"To configure Mailtrap with Symfony Mailer, you need to set the MAILER_DSN environment variable in your .env file. The DSN should follow this format: smtp://username:password@sandbox.smtp.mailtrap.io:2525. You can find your credentials in your Mailtrap inbox settings."

Traduction (français) :

"Pour configurer Mailtrap avec Symfony Mailer, vous devez définir la variable d'environnement MAILER_DSN dans votre fichier .env. Le DSN doit suivre ce format : smtp://nom_utilisateur:mot_de_passe@sandbox.smtp.mailtrap.io:2525. Vous pouvez trouver vos identifiants dans les paramètres de votre boîte de réception Mailtrap."